1. What is Spring WebFlux, and how does its reactive style differ from Spring MVC's approach of using one thread per request?

Spring WebFlux is the reactive web framework in Spring, designed for high concurrency and efficiency. It uses a non-blocking approach, meaning a single thread can handle many requests simultaneously without waiting for long I/O operations (like database calls) to complete. In contrast, Spring MVC uses a synchronous, blocking model where one dedicated thread is tied up and waits for one request to finish before handling the next. This difference allows WebFlux to handle much higher load with fewer resources.

2. In a big Spring application, how do you arrange and handle bean dependencies so the overall structure stays easy to manage?

I arrange beans by following the Single Responsibility Principle (SRP), ensuring each class has one clear job (e.g., separating UserService from UserValidationService). I manage dependencies using Constructor Injection and relying on interfaces for all major services. This ensures loose coupling because components depend on contracts, not concrete implementations, making them easily swappable and testable.

3. If a singleton bean will be used by many threads at the same time, how do you design it so that it stays completely thread-safe?

The core principle is to make the bean stateless, meaning it should not contain any mutable instance variables that change based on the thread accessing it. If state is required, you must use ThreadLocal variables to give each thread its own isolated copy of the data, or use synchronization mechanisms like ReentrantLock or synchronized blocks around the mutable parts.

4. How do you build a modular Spring application where modules don't depend too tightly on each other?

I would enforce modularity by separating the code into independent Maven or Gradle modules. Communication between these modules should only happen through Interfaces (contracts) and Events (asynchronous communication). This prevents the inner implementation details of one module from leaking out and becoming a required dependency for another, thus ensuring loose coupling.

5. From a design point of view, what is the main disadvantage of relying too much on AOP?

The main disadvantage is that AOP introduces hidden behavior that is not visible in the core business logic code. A developer debugging a simple method call may not realize that a complex transaction or security check is being executed by a proxy underneath. This non-obvious coupling can lead to difficult-to-trace bugs and maintenance headaches.

Troubleshooting & Production

6. When Spring Boot's auto-configuration creates hidden bean conflicts, how do you trace the problem and fix the issue inside the auto-config chain?

I trace the conflict by starting the application with the --debug flag, which generates a detailed Auto-Configuration Report. This report shows exactly which auto-config classes considered creating a bean, which conditions failed, and which class ultimately created the conflicting bean. I fix it by creating my own custom bean, which then triggers the @ConditionalOnMissingBean logic in the auto-config, causing the default bean to be skipped.

7. How do you investigate and fix difficult bean creation errors or circular dependencies that only show up during real runtime in large systems?

I start by enabling debug logging for the org.springframework.beans package to get a full trace of the container's resolution process. For circular dependencies, I analyze the output to pinpoint the exact A->B->A loop and break it using a combination of @Lazy or by refactoring the components to use setter injection for one of the dependencies.

8. In production decisions, how do you decide whether to use reactive WebFlux or stick with the normal blocking MVC model?

I choose WebFlux when the application is I/O intensive (e.g., calling many external services or waiting on a slow database) and requires very high concurrency or low latency under heavy load. I stick with the traditional MVC model when the application is CPU-bound (doing heavy calculation) or requires immediate, simple integration with synchronous libraries, as MVC is easier to develop and debug.

9. In a Spring microservices setup, how do you plan and enforce configuration for different environments like dev, QA, stage, and production?

I use Spring Profiles (@Profile) to define environment-specific beans (e.g., using an H2 database in dev vs. Postgres in prod). I separate environment-specific settings into different property files (e.g., application-dev.yml). Finally, I enforce the correct environment by setting the spring.profiles.active variable when deploying to each specific environment (via Kubernetes or command-line flags).